Module 10 – Technical Lecture: Sentiment Analysis (Chapter 26)

The Problem of Sentiment Analysis

Theorical Definition:

```
Given a text corpus D = {d1, d2, ..., dn},
f(di) → {+1, 0, -1}
+1 = Positive, 0 = Neutral, -1 = Negative
```

Opinion Representation Model:

```
(o, f, s, h, t)
o: object, f: feature, s: sentiment, h: holder, t: time
```

This transforms **unstructured text → structured data**.

Example: "I love the battery life of this phone"
o = phone, f = battery life, s = positive, h = user, t = current time

Core Challenges: Context dependence, negation, sarcasm, domain adaptation

Component	Value
Object (o)	phone
Feature (f)	battery life
Sentiment (s)	+1
Holder (h)	user
Time (t)	now

Practical (Python – Basic Sentiment)

```python
from nltk.sentiment import SentimentIntensityAnalyzer

sia = SentimentIntensityAnalyzer()
text = "I love the battery life of this phone"

print(sia.polarity_scores(text))
```

Expected Output:

{'neg': 0.0, 'neu': 0.323, 'pos': 0.677, 'compound': 0.6369}

For the sentence:

"I love the battery life of this phone"

- pos = 0.677 → strong positive sentiment
- neu = 0.323 → some neutral words (e.g., "the", "of")
- neg = 0.0 → no negative sentiment
- compound = 0.6369 → overall positive score

Final Interpretation

Using VADER thresholds:

- compound ≥ 0.05 → Positive

Final Sentiment: Positive

Note:

- Word "love" carries a strong positive score
- Neutral words don't affect sentiment much
- VADER aggregates scores + applies rules

```
print(sia.polarity_scores("I love the battery life but the screen is bad"))
```

- VADER handles **mixed sentiment ("but" rule)**

Key Challenges (with Example)

Challenge	Example
Context shift	"light phone" (+) vs "light sound" (-)
Negation	"not good"
Sarcasm	"Great, it broke again"

Sentiment and Subjectivity Classification

A. Subjectivity Classification

```
g(si) → {subjective, objective}
```

Model (Naive Bayes):

```
P(c|d) ∝ P(c) Π P(wi|c)
```

Calculation Example

Training

Sentence	**Class**
"great product"	subjective
"battery is 3000mAh"	objective

Test:

"great battery"

Compute:

- P(subjective | great, battery)
- P(objective | great, battery)

Higher probability → predicted class

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

texts = ["great product", "battery is 3000mah"]
labels = [1, 0]  # 1=subjective, 0=objective

vec = CountVectorizer()
X = vec.fit_transform(texts)

model = MultinomialNB()
model.fit(X, labels)

test = vec.transform(["great battery"])
print(model.predict(test)
```

B. Document-Level Sentiment

```
TF-IDF(t,d) = TF(t,d) * log(N / df(t))
```

Calculation Example

Corpus:

- D1: "good phone"
- D2: "bad phone"

Word: *phone*

$$TF = 1, df = 2, N = 2$$
$$IDF = \log(2/2) = 0$$

TF-IDF = 0 (common word → low importance)

Classification Function

$$f(d)=\text{sign}(w \cdot d+b)$$

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

texts = ["good phone", "bad phone"]
labels = [1, 0]

tfidf = TfidfVectorizer()
X = tfidf.fit_transform(texts)

model = LogisticRegression()
model.fit(X, labels)

print(model.predict(tfidf.transform(["good"])))
```

C. Sentence-Level Classification

Pipeline: sentence segmentation → subjectivity detection → sentiment classification

Limitation

Ignores cross-sentence meaning.

D. Opinion Lexicon Generation

Dictionary-based and corpus-based methods

```
PMI(w, pos) = log( P(w,pos) / (P(w)P(pos)) )
```

Calculation Example

If:

- P(good, positive) = 0.08
- P(good) = 0.1
- P(positive) = 0.5

$$PMI=\log 0.1\times 0.50.08=\log(1.6)>0$$

"good" is positive

Feature-Based Sentiment Analysis

Goal: Extract (feature, sentiment) pairs

```
S(f) = sum polarity(w)
```

A. Feature Extraction

Frequency-Based

$$score(f) = freq(f)$$

Example:

- battery (10 times) → important

Practical (spaCy)

```
Example: "very good camera" → score = 1.5 * 2 = +3
import spacy
nlp = spacy.load("en_core_web_sm")

doc = nlp("The camera is great but battery is poor")

features = [token.text for token in doc if token.pos_ == "NOUN"]
print(features)
```

B. Sentiment Calculation

$$S(f) = \sum polarity(w)$$

Example

"very good camera"

- good = +2
- very = 1.5

$$S = 1.5 \times 2 = +3$$

C. Negation Handling

$$polarity(w) = -polarity(w)$$

Example:

"not good" → -2

Comparative Sentiment Analysis

```
score(o1, o2) = +1 if o1 preferred, -1 if o2 preferred
```

Example: "A is better than B" → A preferred

Example Calculation

"A is better than B"

$$score(A, B) = +1$$

Practical Rule-Based

```python
def compare(sentence):
    if "better than" in sentence:
        return "first object preferred"
    return "unknown"

print(compare("A is better than B"))
```

Negation Case

"A is not better than B"

Reverse result

Opinion Search and Retrieval

```
R(q,d) = relevance(q,d) + sentiment(d)
```

Model

$$R(q,d) = relevance(q,d) + sentiment(d)$$

Example Calculation

Query:

"best battery"

Document score:

- TF-IDF relevance = 0.7
- Sentiment score = 0.5

$$R = 1.2$$

Opinion Spam and Utility

Spam Detection:

```
h(d) → {spam, genuine}
```

```python
from sklearn.ensemble import RandomForestClassifier

# Example features: length, sentiment
X = [[10, 0.9], [2, 0.1]]
y = [1, 0]

model = RandomForestClassifier()
model.fit(X, y)
```

Utility/Helpfulness Score:

```
U(d) = α*detail + β*readability + γ*relevance
```

Example

- detail = 0.8
- readability = 0.7
- feature coverage = 0.9

$$U = 0.8\alpha + 0.7\beta + 0.9\gamma$$

End-to-End Example

Input: "The screen is amazing but battery is terrible"

```
Output:
{
  "screen": "positive",
  "battery": "negative"
}
```

Step 1: Feature Extraction

- screen
- battery

Step 2: Sentiment Calculation

$$S(screen) = +3, S(battery) = -3$$

Step 3: Output

{

"screen": "positive",

"battery": "negative"

}

Practical Python Example

```
from nltk.sentiment import SentimentIntensityAnalyzer
sia = SentimentIntensityAnalyzer()
print(sia.polarity_scores("The product is great but expensive"))
```

Advanced Topics (Short Lecture Add-on)

1. Transformer Models (BERT)

- Context-aware embeddings
- Handles long dependencies

2. Cross-Domain Adaptation

- Train on one domain → test on another

3. Multilingual Sentiment

- Use multilingual embeddings

In-Class Hands-On Lab

Task 1

Compute sentiment manually using lexicon

Task 2

Train TF-IDF + Logistic Regression

Task 3

Extract features using NLP

Task 4

Detect comparative sentences

Key Takeaways

- Sentiment analysis = **classification + linguistic rules + statistics**
- Feature-based analysis gives **fine-grained insight**
- Real systems combine:
    - Lexicon
    - Machine learning
    - Rule-based logic

What is spaCy?

spaCy is an open-source **Natural Language Processing (NLP) library** in Python designed for fast, production-ready text processing.

Unlike older NLP tools (like NLTK), spaCy is built for:

- Speed (optimized C/Cython backend)
- Production use (industry-ready pipelines)
- Modern NLP tasks (NER, POS tagging, dependency parsing, etc.)

What spaCy Can Do

spaCy provides a complete NLP pipeline:

1. Tokenization

Splits text into words, punctuation, and symbols.

"Apple is great" → ["Apple", "is", "great"]

2. Part-of-Speech (POS) Tagging

Identifies grammatical roles:

- noun
- verb
- adjective
- etc.

Example:

Apple (NOUN), is (VERB), great (ADJ)

3. Named Entity Recognition (NER)

Detects real-world entities:

- Organizations
- People
- Locations
- Dates

Example:

"Apple was founded in California"
→ Apple (ORG), California (GPE)

4. Dependency Parsing

Shows relationships between words:

- subject
- object
- modifiers

5. Lemmatization

Converts words into base form:

running → run
better → good

6. Text Classification (custom models)

You can train spaCy to classify:

- sentiment
- spam detection
- topic labeling

Example Code

```
import spacy

nlp = spacy.load("en_core_web_sm")

doc = nlp("Apple is looking at buying a startup in the UK")

for token in doc:
    print(token.text, token.pos_, token.dep_)
```

spaCy is Popular as:

✓ Very fast compared to NLTK
✓ Pre-trained models available
✓ Easy integration with ML/DL pipelines
✓ Used in real-world applications (chatbots, search engines, analytics systems)

Official Documentation

Here are the most important resources:

- Main documentation:
  https://spacy.io/usage
- Installation guide:
  https://spacy.io/usage#installation
- Language models:
  https://spacy.io/models
- API reference:
  https://spacy.io/api
- Tutorials:
  https://spacy.io/usage/spacy-101

spaCy vs NLTK (quick comparison)

Feature	spaCy	NLTK
Speed	Very fast	Slower
Ease of use	High	Moderate
Production use	Yes	Limited
ML integration	Strong	Basic

spaCy is especially useful for:

- Aspect extraction (noun phrases)
- Named entity extraction
- Preprocessing text before TF-IDF or ML models
- Dependency-based sentiment analysis